UN MANUALE TECNICO DI BM

Il linguaggio bml

Il sottoinsieme tipato di Lua per il Pi Zero W: sintassi, VM a stack, bmvm del kernel e codice nativo ARMv6 A32 con equivalenti C e ASM.

Indice

Perché un linguaggio di bm 3

Una sorgente, quattro bersagli 3

Sintassi e regole di tipo 4

Tipi e letterali 4

Istruzioni e operatori 4

Come funziona il compilatore 5

Tappa 1 e Tappa 2: AST e bytecode a stack 5

Tappa 3 e Tappa 4: `bmvm` del kernel e ARMv6 A32 5

Esempi pratici di codice: BML, C e ASM ARMv6 6

Esempio 1: Somma in un ciclo (`sum`) 6

Esempio 2: Limitazione di un valore (`clamp`) 6

Esempio 3: Massimo comun divisore di Euclide (`gcd`) 7

Esempio 4: Doppia ricorsione (`fib`) 8

Prova passo per passo 10

Punti aperti 11

LEGGIMI

Che cos'è. Come scrivere funzioni calde nel linguaggio tipato di bm (`tools/bml/bml.py`), eseguire la stessa sorgente in Lua standard e ispezionare il bytecode a stack, il codice a registri `bmvm` del kernel, l'equivalente C e le istruzioni macchina native ARMv6 A32 a 32 bit per il Raspberry Pi Zero W.

Come leggerlo. Ogni regola, opcode e parola macchina viene da `tools/bml/bml.py` , `scripts/bmt.py` , `src/bm/bmvm.h` e `docs/design/bml-manual.md` . Il capitolo 4 mostra quattro funzioni complete con sorgente BML, equivalente C e assembly ARMv6 disassemblato fianco a fianco.

Che cosa manca. Ogni tappa aperta della roadmap in `docs/progress/typed.md` è segnata con un'etichetta DA COMPLETARE evidenziata ed elencata nell'ultimo capitolo.

Perché un linguaggio di bm

Un gioco sul Raspberry Pi Zero W 1.1 gira su un singolo core ARM1176JZF-S a 1000 MHz con un budget per frame di 16,7 ms a 60 fps (`docs/PERFORMANCE.md`). Quando il profiler Lua (`profile(true)`) mostra una funzione calda chiamata migliaia di volte per frame, serve la velocità del codice nativo senza mantenere due sorgenti diverse.

Una sorgente, quattro bersagli

Un file `.bml` usa la sintassi di Lua con annotazioni di tipo (`: i32` , `: bool`) sui parametri e sul valore di ritorno. Da un solo file `.bml` , `tools/bml/bml.py` produce:

Bersaglio	Opzione	Dove gira	Scopo
Lua 5.4 standard	`--lua`	`luahost` e la VM Lua della console	Oracolo di riferimento: rimuove `: i32` e `: bool`
VM a stack (Tappa 2)	`--run` / `--dump`	Python sul PC (`StackVM`)	Verifica rapida e ispezione del bytecode
`bmvm` del kernel (Tappa 3)	`--bmvm`	`src/bm/bmvm.c` (`vm_load` , `vm_run`)	Blob sicuro per macchina a registri dentro la cartuccia
ARMv6 A32 nativo (Tappa 4)	`--armv6` / `--dump-armv6`	CPU ARM1176 (`qemu-system-arm` , Pi Zero W)	Parole macchina native a 32 bit (AAPCS)

NOTA

`scripts/bmt.py` traduce la stessa sintassi Lua tipata in C (`--c`) per le funzioni collegate nel kernel (`src/bm/api_collmap.c`). `tools/bml/bml.py` è il compilatore autonomo che genera direttamente bytecode e parole macchina ARMv6 senza un compilatore C.

Sintassi e regole di tipo

Il front end (`Lexer` , `Parser` , `AST` , `TypeChecker` in `tools/bml/bml.py`) controlla ogni istruzione prima di emettere codice. Ogni errore ferma la compilazione e stampa `line <N>: <reason>` .

Tipi e letterali

Tipo	Significato	Letterali	Rappresentazione ARMv6
`i32`	Intero con segno a 32 bit	`0` , `42` , `-2147483648` , `2147483647`	Registro generale a 32 bit (`r0` .. `r10`)
`bool`	Valore di verità stretto	`true` , `false`	`1` (`true`) o `0` (`false`) in un registro a 32 bit

- Le condizioni in `if` , `elseif` , `while` , `and` , `or` e `not` devono avere tipo `bool` . Un `i32` non è mai usato implicitamente come booleano.
- La divisione intera usa `//` e il modulo usa `%` . Entrambi arrotondano verso meno infinito, come in Lua 5.4. L'operatore `/` è rifiutato nella Tappa 1 perché in Lua produce un float.
- Funzioni integrate: `abs(x)` , `min(a, b)` e `max(a, b)` ricevono `i32` e restituiscono `i32` .
- [DA COMPLETARE: la Tappa 6 aggiunge array, struct, la regola definitiva sull'overflow di `i32` e `f32` sul coprocessore VFPv2 dell'ARM1176 (`docs/progress/typed.md`)]

Istruzioni e operatori

Costrutto	Esempio	Regola
Funzione	`function add(a: i32, b: i32): i32 ... end`	Fino a 16 parametri; supporta ricorsione e chiamate a funzioni definite sopra
Variabile locale	`local s = 0` o `local s: i32 = 0`	Visibile fino alla fine del blocco; non può oscurare una variabile attiva
Assegnamento	`s = s + i`	La variabile deve essere dichiarata e avere lo stesso tipo
Condizionale	`if a < b then ... elseif a == b then ... else ... end`	Ogni condizione deve essere `bool`
Ciclo while	`while b ~= 0 do ... end`	La condizione deve essere `bool` ; `break` esce dal ciclo più interno
Ciclo for numerico	`for i = 1, n do ... end` o `for i = n, 1, -2 do ... end`	Estremi e passo opzionale sono `i32`
Operatori	`+` , `-` , `*` , `//` , `%` , `&` , `<` , `<=` , `>` , `>=` , `==` , `~=` , `and` , `or` , `not`	`and` e `or` valutano in corto-circuito come in Lua

Come funziona il compilatore

Tappa 1 e Tappa 2: AST e bytecode a stack

1. `tokenize()` divide il testo sorgente in record `Token(kind, text, line, start, end)` .
2. `Parser` costruisce l'albero sintattico `Program` (`FuncDecl` , `Stmt` , `Expr`) e annota gli intervalli di byte di `: i32` e `: bool` affinché `to_lua()` possa rimuoverli.
3. `TypeChecker` verifica gli ambiti delle variabili, i tipi degli operatori, le chiamate di funzione e controlla che ogni percorso di una funzione con tipo di ritorno termini con `return` .
4. `BytecodeCompiler` emette le istruzioni a stack (`PUSH_CONST` , `LOAD` , `STORE` , `ADD` , `SUB` , `MUL` , `IDIV` , `IMOD` , `BAND` , `LE` , `JZ` , `JF_KEEP` , `JT_KEEP` , `CALL` , `RET`).

Tappa 3 e Tappa 4: `bmvm` del kernel e ARMv6 A32

1. `compute_stack_depths()` calcola l'altezza esatta dello stack di valutazione prima di ogni istruzione.
2. `BMVMCompiler` mappa i parametri in `R[0..nparams-1]` , le costanti in `R[nparams..nparams+nconst-1]` , i locali in `R[nparams+nconst..]` e gli slot dello stack `S[k]` sopra i locali, generando un blob `BMV1` per `src/bm/bmvm.c` .
3. `ARMv6Compiler` emette parole macchina ARMv6 A32 a 32 bit secondo l'ABI AAPCS (`r0` .. `r3` argomenti, `r0` risultato, registri callee-saved `r4` .. `r10` per i primi 7 slot locali/stack e slot `[fp, #-off]` sullo stack per le variabili eccedenti).
4. [DA COMPLETARE: la Tappa 5 carica il blob binario ARMv6 in RAM eseguibile sul Pi Zero W con pulizia della D-cache e invalidazione della I-cache (`native_load` , `native_run` in `docs/progress/typed.md`)]

Esempi pratici di codice: BML, C e ASM ARMv6

Esempio 1: Somma in un ciclo (`sum`)

Il ciclo canonico di `tests/bml/sum.bml` somma gli interi da `1` a `n` .

```
-- 1. Sorgente BML (tests/bml/sum.bml)
function sum(n: i32): i32
  local s = 0
  for i = 1, n do s = s + i end
  return s
end
```

```
/* 2. Equivalent C (scripts/bmt.py --c) */
int32_t bmt_sum(int32_t n)
{
    int32_t s = 0;
    {
        int32_t i__end = n;
        for (int32_t i = 1; i <= i__end; i++) {
            s = (s + i);
        }
    }
    return s;
}
```

```
; 3. ARMv6 A32 leaf machine code (9 words = 36 bytes, r0 = n, r1 = s, r2 = i)
  0x0000: E3A01000    mov   r1, #0          ; s = 0
  0x0004: E3A02001    mov   r2, #1          ; i = 1
  0x0008: E1520000    cmp   r2, r0          ; compare i with n
  0x000c: CA000002    bgt   0x1c            ; if i > n jump to 0x1c
  0x0010: E0811002    add   r1, r1, r2      ; s = s + i
  0x0014: E2822001    add   r2, r2, #1      ; i = i + 1
  0x0018: EAFFFFFA    b     0x08            ; repeat loop
  0x001c: E1A00001    mov   r0, r1          ; return s in r0
  0x0020: E12FFF1E    bx    lr              ; return to caller
```

Esempio 2: Limitazione di un valore (`clamp`)

`clamp` limita `x` all'intervallo `[lo, hi]` usando le funzioni integrate `min` e `max` , che su ARMv6 diventano mosse condizionali (`movlt` , `movgt`) senza salti.

```
-- 1. Sorgente BML (tests/bml/features.bml)
function clamp(x: i32, lo: i32, hi: i32): i32
  return min(max(x, lo), hi)
end
```

```
/* 2. Equivalent C */
int32_t bmt_clamp(int32_t x, int32_t lo, int32_t hi)
{
    return bmt_min_i32(bmt_max_i32(x, lo), hi);
}
```

```
; 3. ARMv6 A32 machine code (r4 = x, r5 = lo, r6 = hi, r7/r8 = stack slots)
  0x004c: E92D4FF0    push  {r4-r10, fp, lr}
  0x0050: E1A0B00D    mov   fp, sp
  0x0054: E3A0C004    mov   ip, #4
  0x0058: E04DD00C    sub   sp, sp, ip
  0x005c: E1A04000    mov   r4, r0          ; r4 = x
  0x0060: E1A05001    mov   r5, r1          ; r5 = lo
  0x0064: E1A06002    mov   r6, r2          ; r6 = hi
  0x0068: E1A07004    mov   r7, r4          ; S[0] = x
  0x006c: E1A08005    mov   r8, r5          ; S[1] = lo
  0x0070: E1570008    cmp   r7, r8          ; compare x with lo
  0x0074: B1A07008    movlt r7, r8          ; max(x, lo): if x < lo then r7 = lo
  0x0078: E1A08006    mov   r8, r6          ; S[1] = hi
  0x007c: E1570008    cmp   r7, r8          ; compare max(x, lo) with hi
  0x0080: C1A07008    movgt r7, r8          ; min(..., hi): if r7 > hi then r7 = hi
  0x0084: E1A00007    mov   r0, r7          ; result in r0
  0x008c: E1A0D00B    mov   sp, fp
  0x0090: E8BD8FF0    pop   {r4-r10, fp, pc}
```

Esempio 3: Massimo comun divisore di Euclide (`gcd`)

`gcd` usa `abs` , un ciclo `while` e `%` . Poiché l'ARM1176 non ha un'istruzione hardware per la divisione intera, `a % b` chiama la routine A32 di 30 parole `__bml_idivmod` , che restituisce il quoziente floor in `r0` e il modulo floor in `r1` .

```
-- 1. Sorgente BML (tests/bml/features.bml)
function gcd(a: i32, b: i32): i32
  a = abs(a)
  b = abs(b)
  while b ~= 0 do
    local r = a % b
    a = b
    b = r
  end
  return a
end
```

```
/* 2. Equivalent C */
int32_t bmt_gcd(int32_t a, int32_t b)
{
    a = abs(a);
    b = abs(b);
    while ((b != 0)) {
        int32_t r = bmt_imod(a, b);
        a = b;
        b = r;
    }
    return a;
}
```

```
; 3. ARMv6 A32 machine code (r4 = a, r5 = b, r6 = r)
  0x00ac: E1A07004    mov   r7, r4
  0x00b0: E3570000    cmp   r7, #0
  0x00b4: B2677000    rsblt r7, r7, #0      ; abs(a) via reverse-subtract if < 0
  0x00b8: E1A04007    mov   r4, r7
  ...
; while b ~= 0 loop body:
  0x00f0: E1A00007    mov   r0, r7          ; r0 = a
  0x00f4: E1A01008    mov   r1, r8          ; r1 = b
  0x00f8: EB0001B5    bl    __bml_idivmod   ; r0 = a // b, r1 = a % b
  0x00fc: E1A07001    mov   r7, r1          ; S[0] = remainder r1
  0x0100: E1A06007    mov   r6, r7          ; local r = a % b
  0x0104: E1A07005    mov   r7, r5
  0x0108: E1A04007    mov   r4, r7          ; a = b
  0x010c: E1A07006    mov   r7, r6
  0x0110: E1A05007    mov   r5, r7          ; b = r
  0x0114: EAFFFFEC    b     0x00cc          ; jump back to while test
```

Esempio 4: Doppia ricorsione (`fib`)

In `fib(n - 1) + fib(n - 2)` , la prima chiamata ricorsiva lascia il suo risultato in `S[0]` , che è mappato nel registro callee-saved `r5` . Quando parte la seconda `bl fn_fib` , `fn_fib` salva e ripristina `r4` .. `r10` sul proprio stack frame, così `r4` (`n`) e `r5` (`fib(n - 1)`) restano intatti senza accessi aggiuntivi in memoria.

```
-- 1. Sorgente BML (tests/bml/features.bml)
function fib(n: i32): i32
  if n <= 0 then
    return 0
  elseif n == 1 then
    return 1
  else
    return fib(n - 1) + fib(n - 2)
  end
end
```

```
/* 2. Equivalent C */
int32_t bmt_fib(int32_t n)
{
    if ((n <= 0)) return 0;
    else if ((n == 1)) return 1;
    else return (bmt_fib((n - 1)) + bmt_fib((n - 2)));
}
```

```
; 3. ARMv6 A32 recursive else branch (r4 = n, r5 = S[0], r6 = S[1])
  0x0198: E1A05004    mov   r5, r4          ; S[0] = n
  0x019c: E3A06001    mov   r6, #1          ; S[1] = 1
  0x01a0: E0455006    sub   r5, r5, r6      ; S[0] = n - 1
  0x01a4: E1A00005    mov   r0, r5          ; arg0 = n - 1
  0x01a8: EBFFFFDF    bl    fn_fib          ; call fib(n - 1)
  0x01ac: E1A05000    mov   r5, r0          ; r5 (callee-saved) = fib(n - 1)
  0x01b0: E1A06004    mov   r6, r4          ; r6 = n (r4 preserved across call)
  0x01b4: E3A07002    mov   r7, #2          ; r7 = 2
  0x01b8: E0466007    sub   r6, r6, r7      ; r6 = n - 2
  0x01bc: E1A00006    mov   r0, r6          ; arg0 = n - 2
  0x01c0: EBFFFFD9    bl    fn_fib          ; call fib(n - 2) (r5 preserved)
  0x01c4: E1A06000    mov   r6, r0          ; r6 = fib(n - 2)
  0x01c8: E0855006    add   r5, r5, r6      ; r5 = fib(n - 1) + fib(n - 2)
  0x01cc: E1A00005    mov   r0, r5          ; return result in r0
```

Prova passo per passo

PROVA

Compila `tests/bml/sum.bml`, ispeziona il bytecode a stack e le 9 parole macchina ARMv6, eseguilo nella VM e lancia la suite completa sulla CPU ARM1176 emulata:

1. Esegui `python3 tools/bml/bml.py tests/bml/sum.bml --dump --run sum 10`: vedi le 21 istruzioni a stack e il risultato `55`. 2. Esegui `python3 tools/bml/bml.py tests/bml/sum.bml --dump-armv6`: vedi le 9 parole esadecimali A32 da `0x0000` a `0x0020`. 3. Esegui `make test-bml`: la suite controlla 23 programmi errati, `sum`, `features` e 200 programmi casuali confrontando Lua, `StackVM`, `bmvm` in C e `qemu-system-arm -cpu arm1176`.

DA COPIARE

```
python3 tools/bml/bml.py tests/bml/sum.bml --dump --run sum 10
python3 tools/bml/bml.py tests/bml/sum.bml --dump-armv6
python3 tools/bml/bml.py tests/bml/features.bml --bmvm build/features.bmv
make test-bml
```

Punti aperti

- **Sintassi e regole di tipo**: la Tappa 6 aggiunge array, struct, la regola definitiva sull'overflow di `i32` e `f32` sul coprocessore VFPv2 dell'ARM1176 (`docs/progress/typed.md`)
- **Come funziona il compilatore**: la Tappa 5 carica il blob binario ARMv6 in RAM eseguibile sul Pi Zero W con pulizia della D-cache e invalidazione della I-cache (`native_load` , `native_run` in `docs/progress/typed.md`)